UNIVERSITY OF THE PEOPLE

CS 2301-01 - AY2027-T1

Unit 1 Assignment

Submitted by: Mubashir Ali

Instructor: Joseph Ego

Date: September 11, 2026

Multithreading, CPU Scheduling, and Context Switching in a Hospital Management System

Introduction

A large-scale hospital management system must process many activities at the same time, including patient registration, electronic health records, laboratory results, medication orders, emergency alerts, billing, appointment scheduling, and communication between departments. A single-threaded process model combined with non-preemptive CPU scheduling can cause delays because one task may retain the CPU while other tasks wait. Multithreading and pre-emptive CPU scheduling can improve responsiveness by allowing the operating system to manage multiple tasks more efficiently. However, these improvements depend on effective context switching and careful resource management. Multithreading allows a process to contain multiple threads of execution that can perform different tasks concurrently, while CPU scheduling determines which ready process or thread receives CPU time (Hailperin, 2019; Silberschatz et al., 2018).

1. Impact of Multithreading and Context Switching

1.1 How Multithreading Enhances Hospital System Performance

Multithreading can significantly improve the performance and responsiveness of the hospital management system because different activities can be divided into separate threads instead of forcing the entire application to process one task at a time. A thread is a basic unit of CPU utilization with its own execution state, while threads belonging to the same process can share code, data, and other resources. This makes multithreading particularly useful for applications that must handle many concurrent activities (Silberschatz et al., 2018).

For example, the hospital management application could use separate threads for patient registration, database operations, laboratory-result processing, emergency notifications, and user-interface requests. If a laboratory thread is waiting for data from a laboratory device or database, the CPU can execute another ready thread instead of leaving the application unresponsive. Similarly, a doctor using the electronic health record interface should still be able to view a patient's information while another thread processes background database operations.

Multithreading is especially important for critical hospital processes. An emergency-alert thread could monitor urgent patient conditions, while another thread handles routine appointment scheduling. A medication-order thread could process prescriptions while a reporting thread generates administrative reports. On a multicore processor, independent threads may also execute in parallel, potentially increasing throughput. OSTEP explains that scheduling policies must balance performance and fairness, while multithreaded execution provides an important mechanism for handling concurrent work (Arpaci-Dusseau & Arpaci-Dusseau, 2018).

Therefore, multithreading improves responsiveness, resource utilization, and the ability of the hospital system to handle multiple users and operations simultaneously. It does not eliminate the need for scheduling; instead, it creates more executable units that the operating system must manage efficiently.

1.2 Role of Context Switching

Context switching allows the operating system to move the CPU from one executing process or thread to another. During a context switch, the current execution state is saved and the state of another task is restored so that the interrupted task can continue later. The saved context can include processor registers, program-counter information, and process or thread state (Silberschatz et al., 2018).

Consider a hospital server processing three tasks: updating an electronic health record, receiving an emergency alert, and generating a billing report. If the billing thread is executed when an urgent alert becomes ready, a pre-emptive scheduler can interrupt the billing thread, save its context, and allow the emergency-alert thread to execute. After the urgent work receives CPU time, the operating system can restore the billing thread's previous state and continue its execution.

Context switching therefore provides the mechanism that makes multitasking possible. However, it also introduces overhead because the CPU performs saving and restoring operations instead of useful application work. Excessive context switching can reduce performance. Consequently, a hospital system should use scheduling policies and time slices that provide responsiveness without creating unnecessary switching overhead.

2. Comparison of Scheduling Algorithms

2.1 Pre-emptive and Non-Pre-emptive Scheduling in Hospital Operations

Pre-emptive scheduling allows the operating system to interrupt a currently running process or thread and assign the CPU to another ready task. This approach is valuable when some hospital operations are more time-sensitive than others. For example, suppose a long-running report-generation task is using the CPU when an emergency patient-monitoring task becomes ready. A pre-emptive scheduler can interrupt the report task and give CPU time to the emergency task. This can reduce response time for critical operations.

Round Robin and Shortest Time-to-Completion First (STCF), also called Shortest Remaining Time First in many textbooks, are examples of pre-emptive scheduling approaches. OSTEP explains that STCF adds pre-emption to Shortest Job First by selecting the job with the smallest remaining execution time, while Round Robin repeatedly gives runnable jobs a limited time slice (Arpaci-Dusseau & Arpaci-Dusseau, 2018).

Non-pre-emptive scheduling, in contrast, does not forcibly remove the CPU from a running task. Once a process receives the CPU, it normally continues until it terminates or blocks for an operation such as I/O. First-Come, First-Served (FCFS/FIFO) and traditional Shortest Job First (SJF) are examples of non-pre-emptive approaches. Hailperin (2019) explains scheduling as the mechanism through which an operating system determines which concurrent computation should receive attention.

In the hospital environment, non-pre-emptive scheduling may work adequately for predictable background tasks, such as batch report generation. However, it can be problematic when a long-running task delays an urgent operation. Pre-emptive scheduling is generally more suitable for interactive and time-sensitive hospital applications because it can improve response time. Its disadvantage is additional context-switching overhead and greater scheduling complexity.

2.2 Resource Allocation and Scheduling Methods

The two scheduling approaches also affect how CPU resources are allocated. Under non-pre-emptive scheduling, a task that receives the CPU may retain it for a relatively long period. This approach is simple and can reduce forced context switches, but it can produce the convoy effect, where short tasks remain behind a long task. In a hospital system, a lengthy data-analysis operation could delay a small but important patient-information request.

The major advantage of non-pre-emptive scheduling is simplicity and lower pre-emption overhead. It can be appropriate for background workloads where immediate response is not essential. Its disadvantages include poor responsiveness, possible long waiting times, and difficulty handling urgent work.

Pre-emptive scheduling provides more flexible CPU allocation. The operating system can interrupt a lower-priority or currently executing task and allocate CPU time to a more urgent task. For example, an emergency monitoring process could receive CPU time before a routine report-generation process. This improves responsiveness and allows hospital applications to prioritize time-sensitive work.

However, pre-emption introduces context-switching overhead. If the system switches too frequently between threads, CPU time can be consumed by scheduling rather than useful computation. OSTEP identifies this broader

trade-off between scheduling performance and fairness: a scheduling policy that improves one measure may negatively affect another (Arpaci-Dusseau & Arpaci-Dusseau, 2018).

For this reason, the hospital should use pre-emptive scheduling for interactive and critical workloads while appropriately managing priorities and time slices for background processes.

3. Context Switching Plan

3.1 Applying Context Switching to Critical Hospital Processes

A practical context-switching plan can divide the hospital management system into several concurrent processes or threads:

1. **Emergency monitoring:** highest priority because it must respond rapidly to urgent events.
2. **Patient-care database operations:** high priority because doctors and nurses require timely access to patient information.
3. **Laboratory and diagnostic processing:** medium-to-high priority because results can influence clinical decisions.
4. **Medication and prescription processing:** high priority because delays may affect patient treatment.
5. **Billing and administrative reporting:** lower priority because these tasks are generally less time sensitive.

Suppose the CPU is processing a routine administrative report when an emergency monitoring thread becomes ready. The scheduler can pre-empt the report thread and perform a context switch. The operating system saves the report thread's execution state and loads the emergency thread's context. The emergency task then receives CPU time. When the urgent task blocks or finishes, the scheduler can restore the report thread and continue from its saved state.

This process can be repeated as different hospital tasks become ready. The important principle is that context switching should be driven by the system's scheduling policy rather than occurring unnecessarily.

3.2 System Stability and Efficiency

Context switching contributes to system stability because it allows multiple activities to make progress without losing their execution states. Without saving and restoring context correctly, an interrupted task could lose information about where it was executing, potentially producing incorrect results. The process control information maintained by the operating system provides the information necessary to resume execution correctly (Silberschatz et al., 2018).

For example, if a doctor is updating a patient's electronic record and the system temporarily switches to an emergency notification task, the patient's record update must resume from the correct execution state after the emergency operation receives CPU time. Similarly, a laboratory-processing thread must continue from its previous state after another task temporarily uses the CPU.

Efficiency requires balance. Too few context switches can make the system unresponsive, while excessive context switching creates overhead. The hospital should therefore combine multithreading with an appropriate pre-

emptive scheduling policy, meaningful priorities, and carefully selected time slices. Critical processes should receive rapid access to CPU resources, while lower-priority background activities should continue to receive sufficient CPU time to prevent indefinite starvation.

Conclusion

Upgrading the hospital management system from a single-threaded, non-pre-emptive model to a multithreaded and pre-emptive architecture can substantially improve responsiveness and multitasking. Multithreading allows different hospital functions to execute concurrently, while pre-emptive scheduling enables the operating system to respond rapidly when urgent tasks require CPU time. Context switching connects these mechanisms by saving the state of one task and restoring another, allowing multiple processes and threads to share CPU resources safely.

The upgrade must nevertheless be designed carefully. Multithreading can improve concurrency and processor utilization, but excessive synchronization and context switching can introduce overhead. Similarly, pre-emptive scheduling can improve response time but requires more sophisticated resource management. For a hospital environment, the most appropriate strategy is therefore to prioritize critical clinical operations while allowing routine administrative workloads to execute in the background. With effective scheduling and controlled context switching, the system can provide better responsiveness, resource utilization, and reliability for both healthcare professionals and patients.

References

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces* (Version 1.10). University of Wisconsin–Madison. https://pages.cs.wisc.edu/~remzi/OSTEP/

Hailperin, M. (2019). *Operating systems and middleware: Supporting controlled interaction* (Revised ed. 1.3.1). Gustavus Adolphus College. https://www.cs.utexas.edu/~ans/classes/cs439/osm-rev1.3.1.pdf

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System concepts* (10th ed.). Wiley. Operating System Concepts, 10th Edition